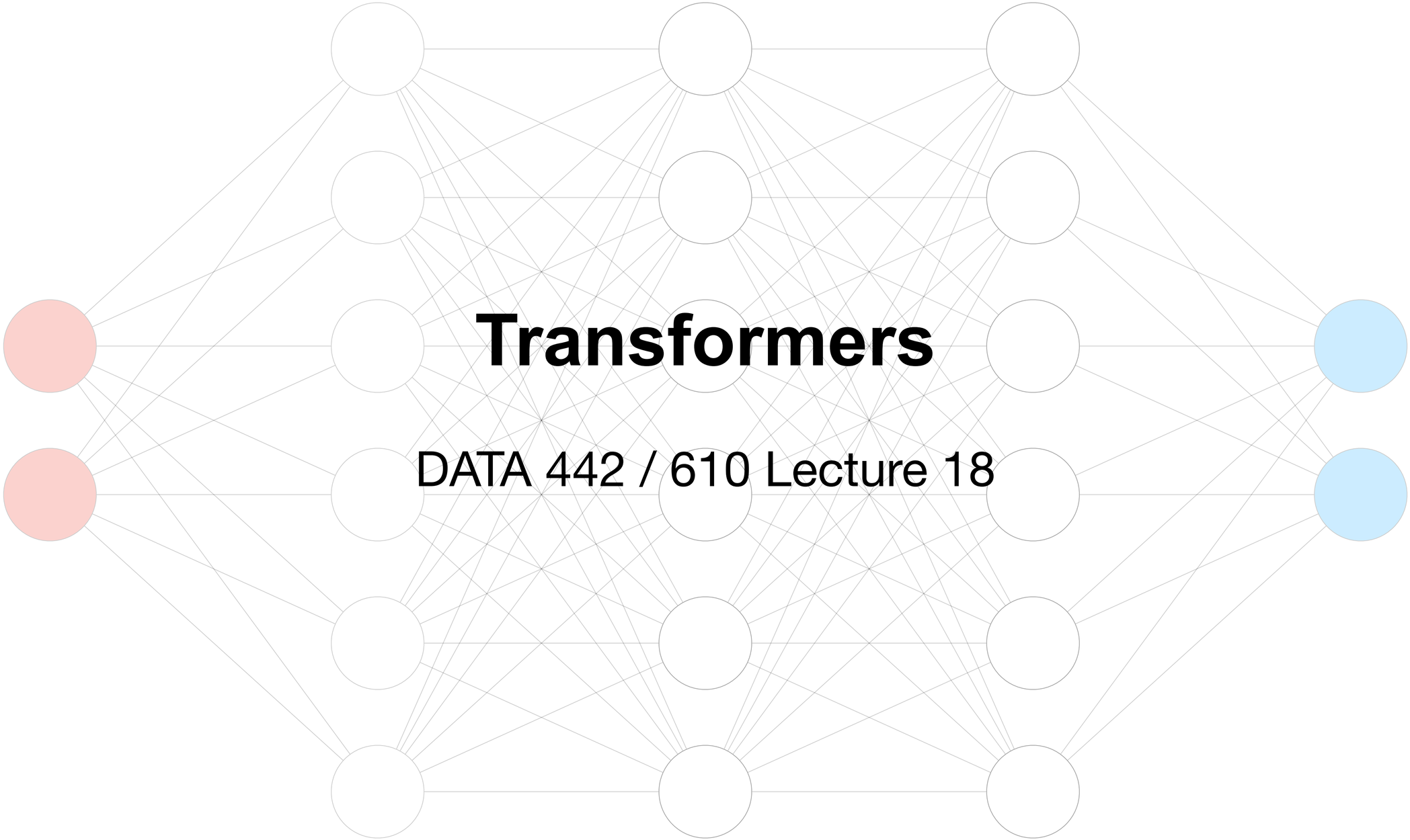
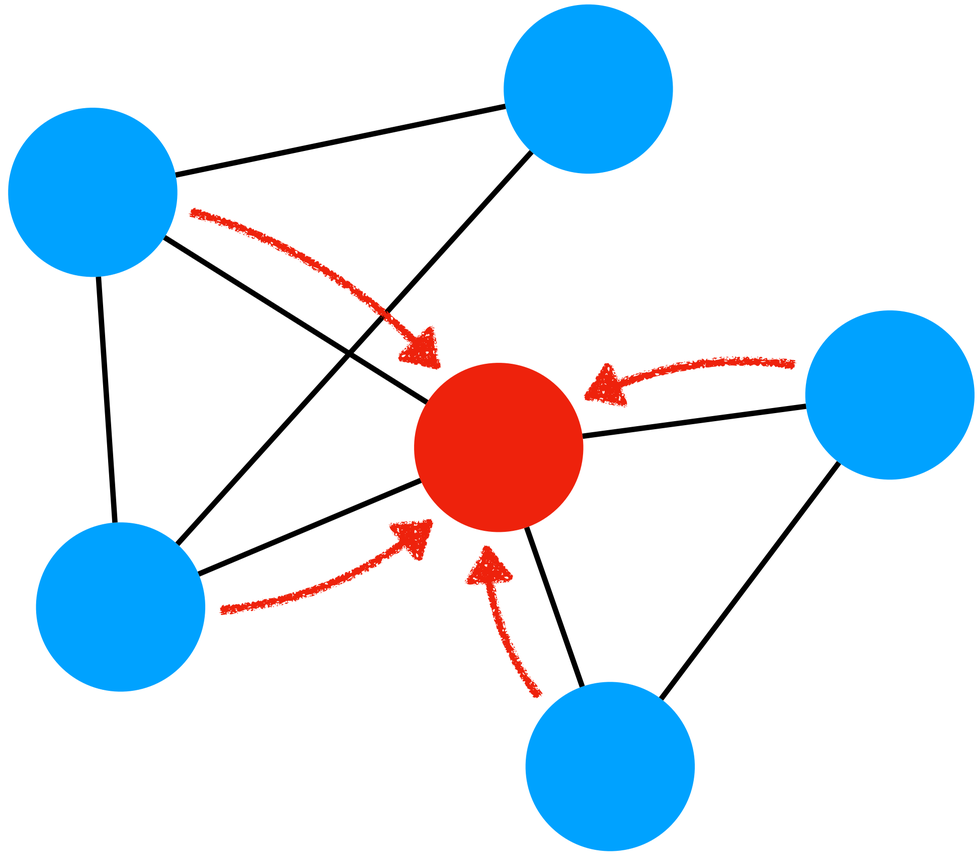


Transformers

DATA 442 / 610 Lecture 18

Graph convolution



Graph convolution operations generally involve two steps:

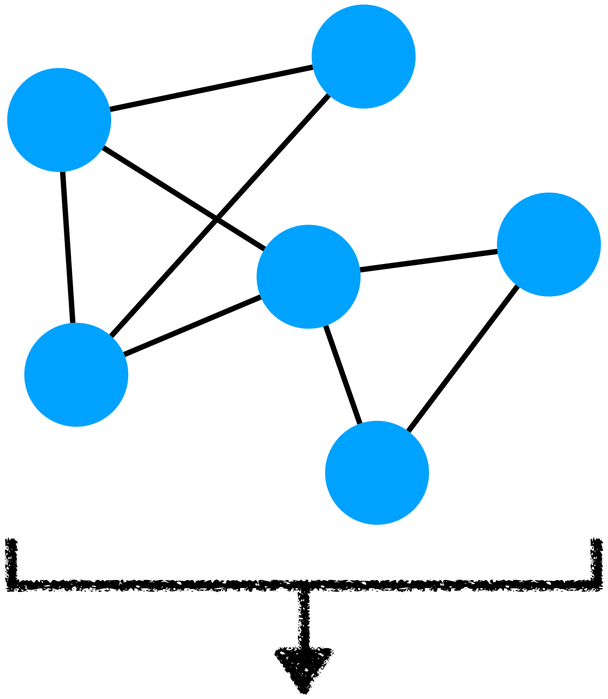
1. **Message passing:** nodes send and receive messages from their neighbors

$$\mathbf{m}_i = \sum_{j \in \mathcal{N}(i)} M(\mathbf{x}_i, \mathbf{x}_j, e_{ij})$$

2. **Node update:** nodes update their features based on incoming messages

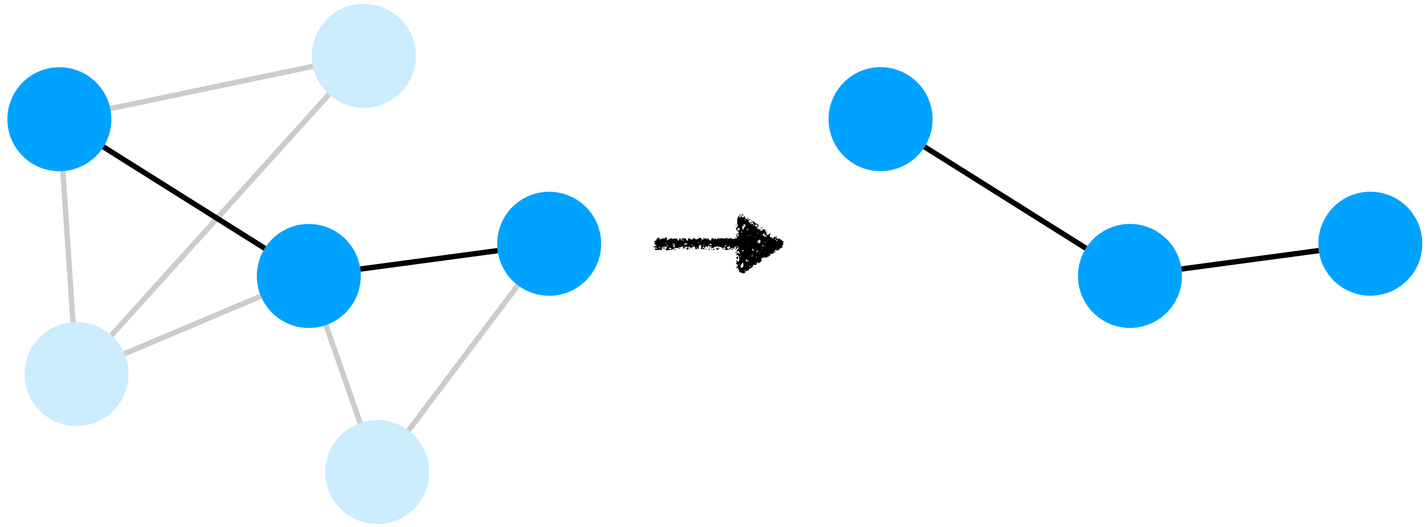
Graph pooling

Global pooling

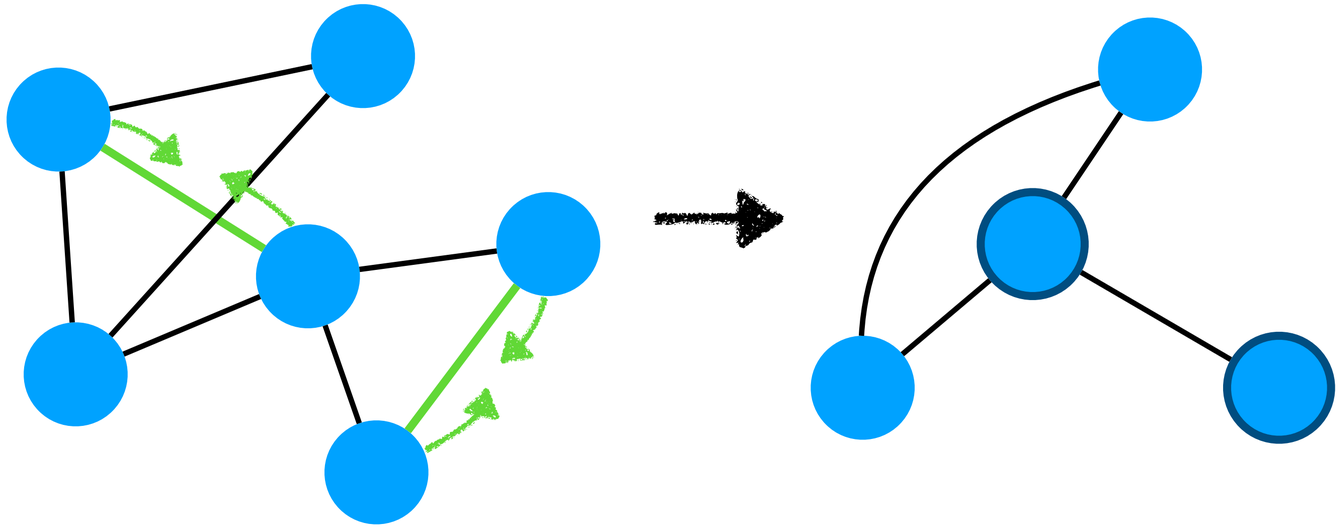


max, mean, sum, min, …

Local (top-k) pooling

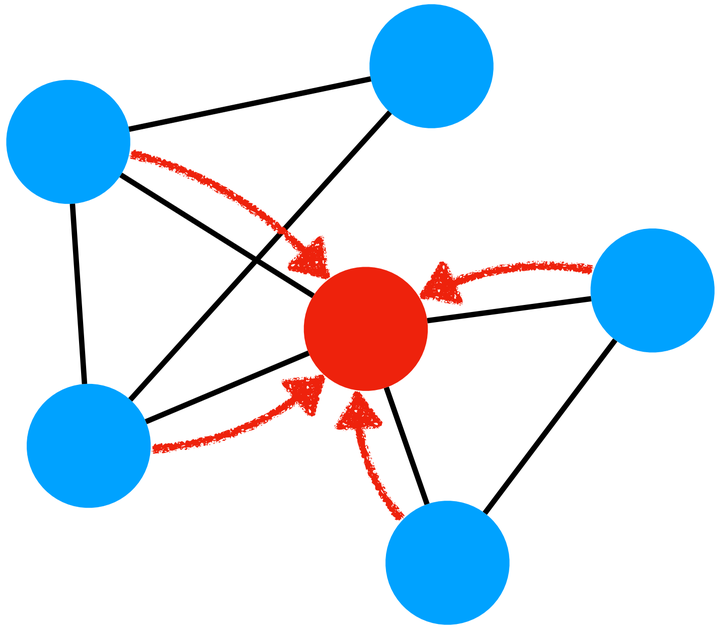


Edge contraction

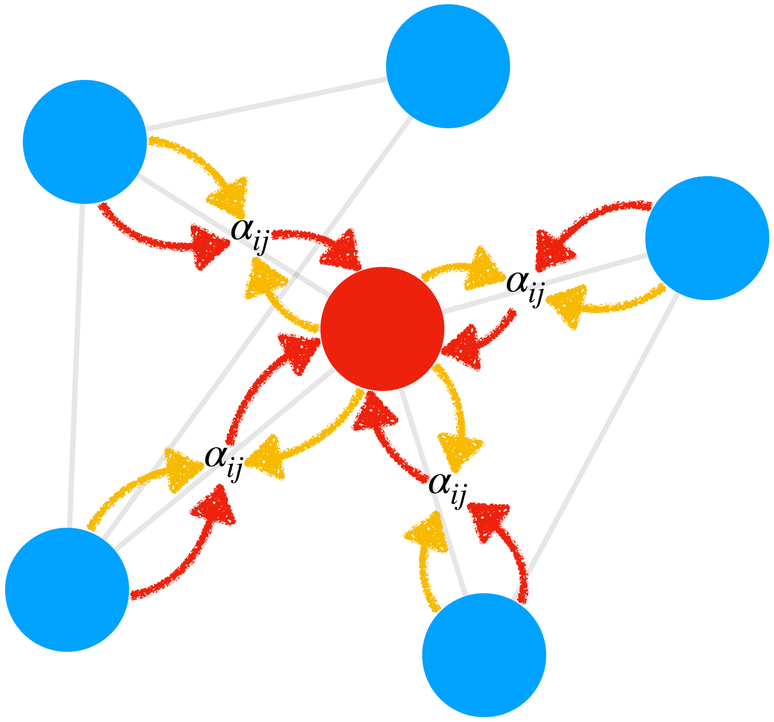


Graph operations

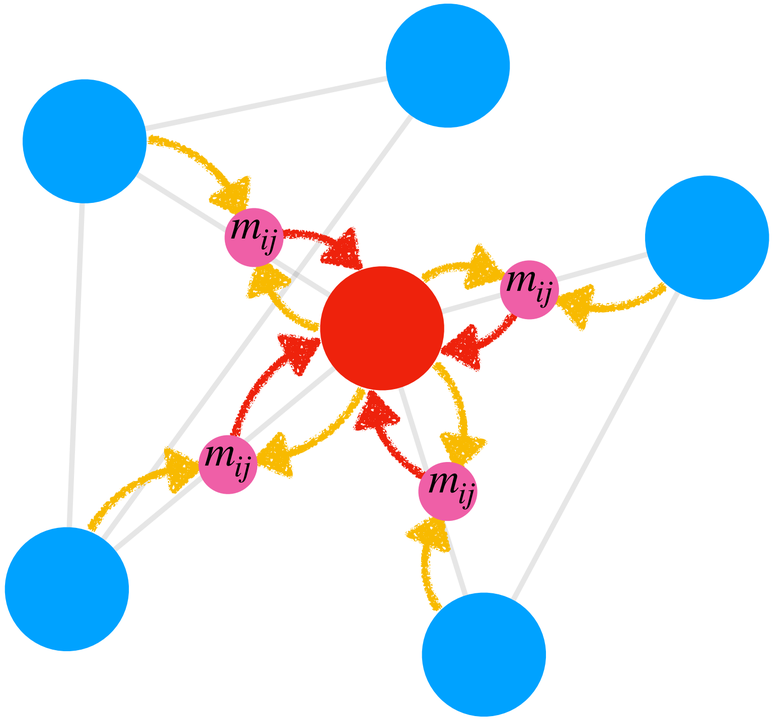
Convolutional



Attentional



Message passing



Outline

- Review of RNN and Seq2Seq
- Review of Attention
- Transformer model

Relevant reading: ArXiv:1706.03762 Blog post

Long Short Term Memory

Gating functions

$$i^{(t)} = \sigma\left(W_{ii}x^{(t)} + b_{ii} + W_{hi}h^{(t-1)} + b_{hi}\right)$$

$$f^{(t)} = \sigma\left(W_{if}x^{(t)} + b_{if} + W_{hf}h^{(t-1)} + b_{hf}\right)$$

$$o^{(t)} = \sigma\left(W_{io}x^{(t)} + b_{io} + W_{ho}h^{(t-1)} + b_{ho}\right)$$

Memory cell input

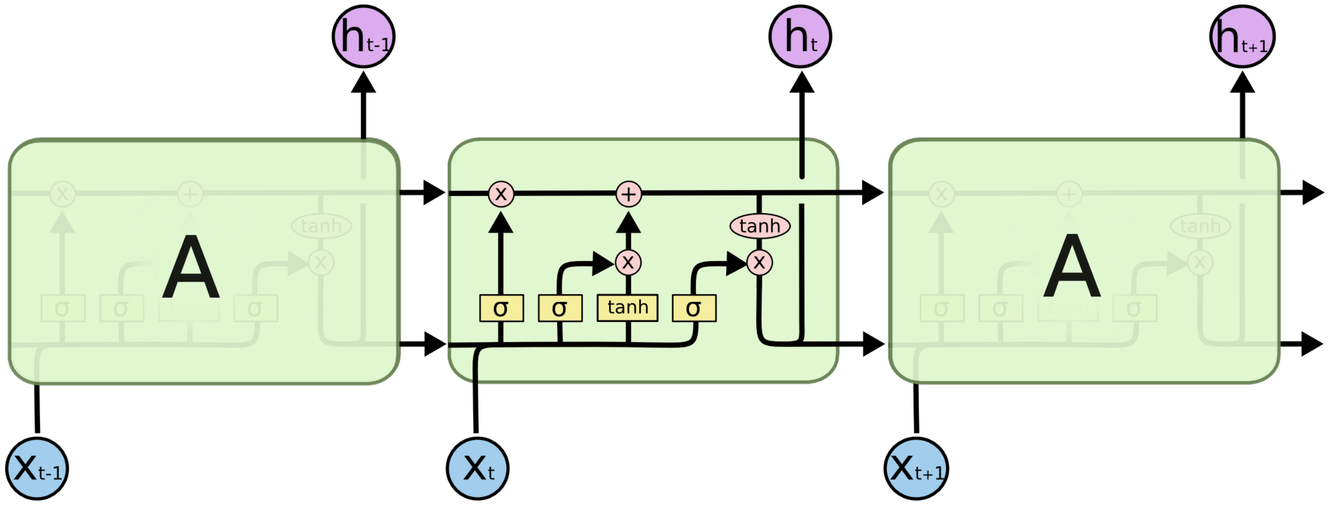
$$g^{(t)} = \tanh\left(W_{ig}x^{(t)} + b_{ig} + W_{hg}h^{(t-1)} + b_{hg}\right)$$

Memory cell update

$$c^{(t)} = f^{(t)}c^{(t-1)} + i^{(t)}g^{(t)}$$

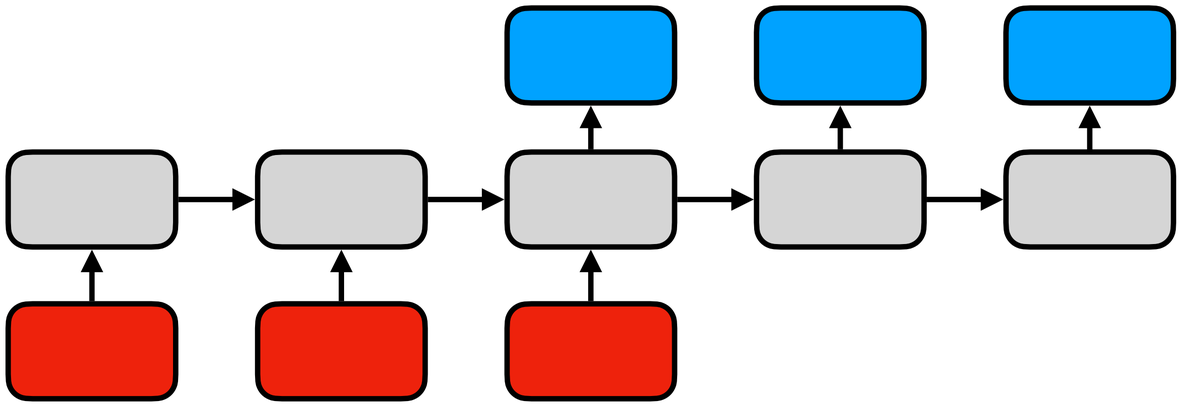
Hidden layer update

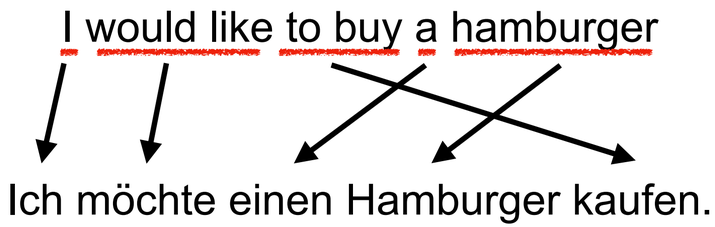
$$h^{(t)} = o^{(t)}\tanh c^{(t)}$$



Gating and a memory cell enable selective preservation of long-range information

More RNN Problems



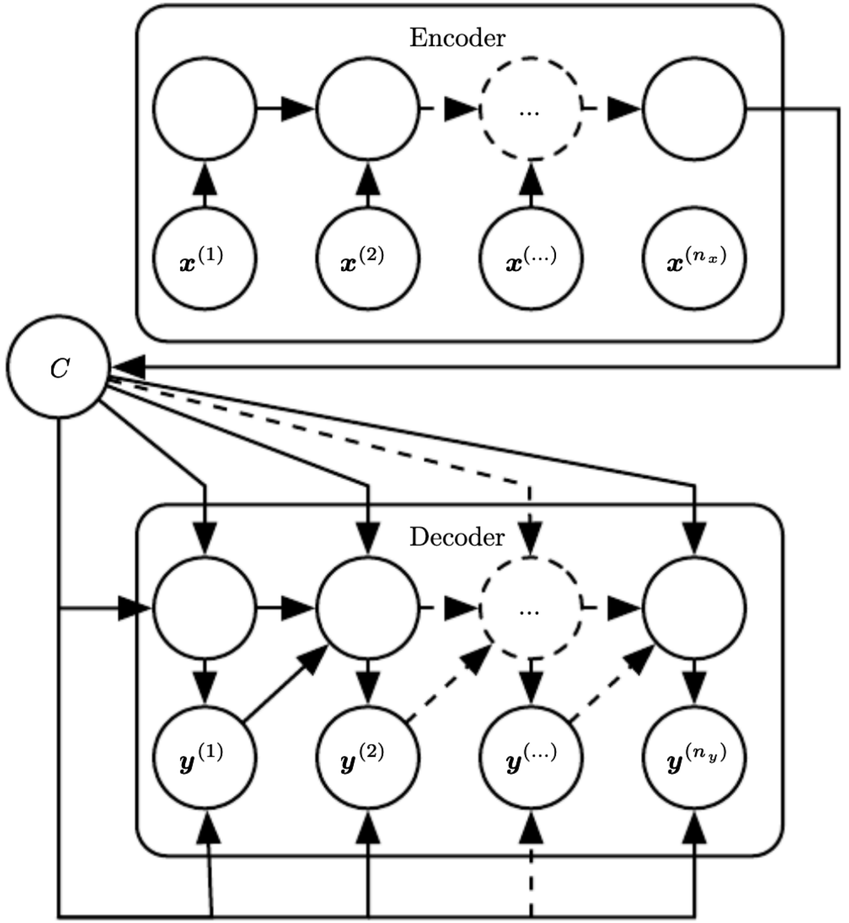


Variable sequence order can carry same information

RNN for machine translation: Seq2seq

- Sequence to sequence aims to generate an arbitrary-length output from an arbitrary-length input
- An **encoder** transforms the input sequence into a **context variable** that holds relevant information
- A **decoder** uses the context variable to generate an output sequence
- Encoder/decoder trained to jointly maximize

$$\log p\left(y^{(1)}, \ldots, y^{(n)} \mid x^{(1)}, \ldots, x^{(m)}\right)$$



RNN + Attention

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION
BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

ArXiv.1409.0473

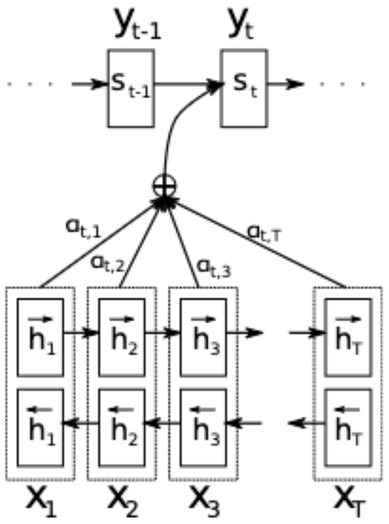


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \ldots, x_T)$.

Encoder: bi-directional RNN

Hidden state for ith word: $h_i = \left\{ \overrightarrow{h_i}, \overleftarrow{h_i} \right\}$

Decoder: RNN with context vector

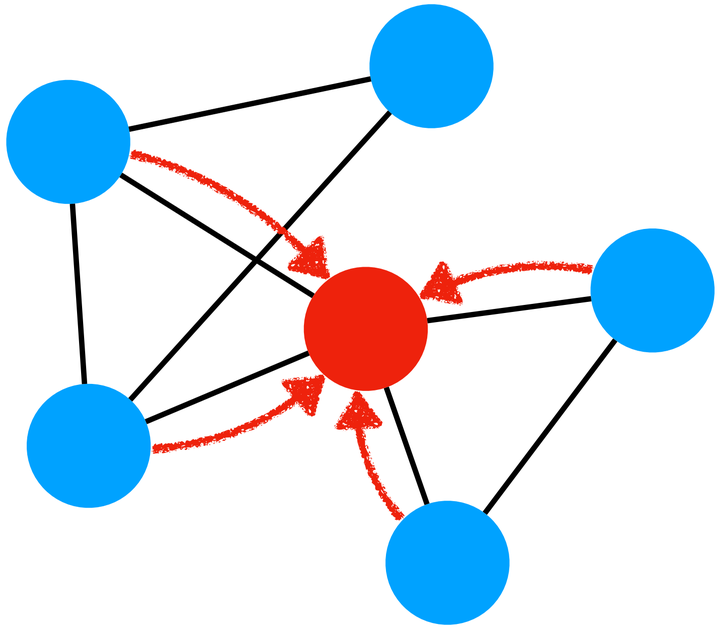
$$p(y_n \mid y_1, \ldots, y_{n-1}; h_n) = f\left(y_{n-1}, s_n, c_n(h_n)\right)$$

Context vector: unique for each word

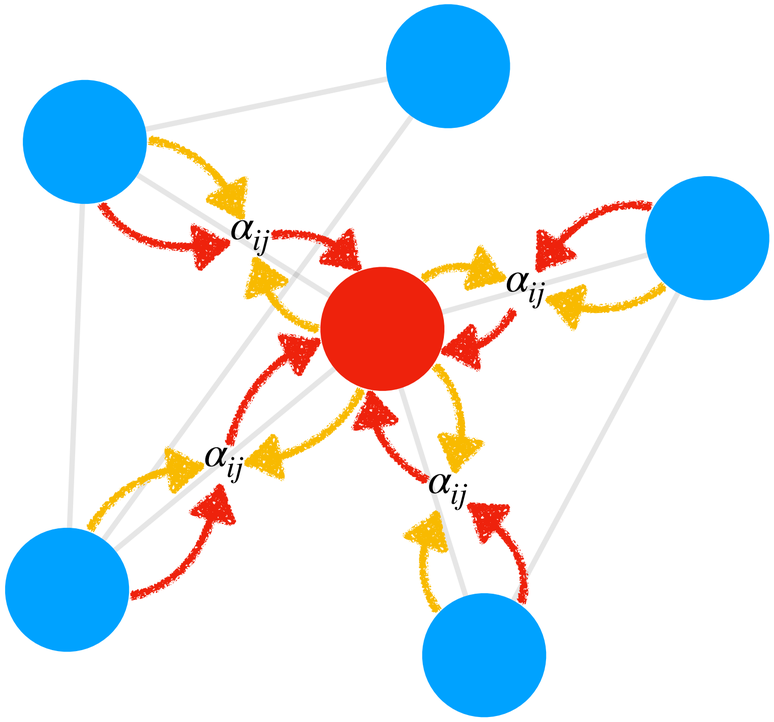
$$c_i = \sum_{j=1}^{t} \alpha_{ij} h_j = \sum_{j=1}^{t} \frac{\exp\left(a(s_{i-1}, h_j)\right)}{\sum_k \exp\left(a(s_{i-1}, h_k)\right)} h_j$$

Attention

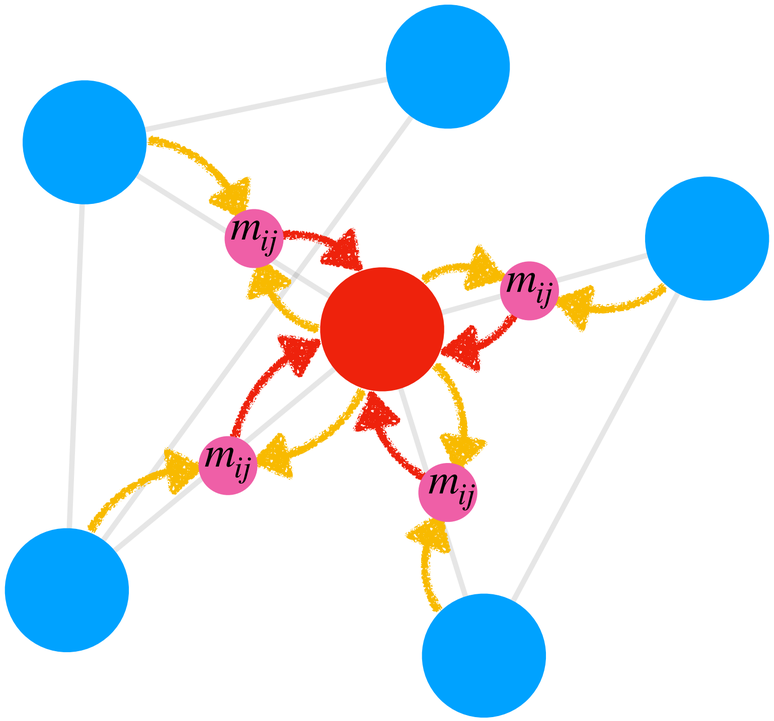
Convolutional



Attentional



Message passing



Attention

Scaled dot-product attention

Query: $Q = W_Q x_i$

Key: $K = W_K x_j$

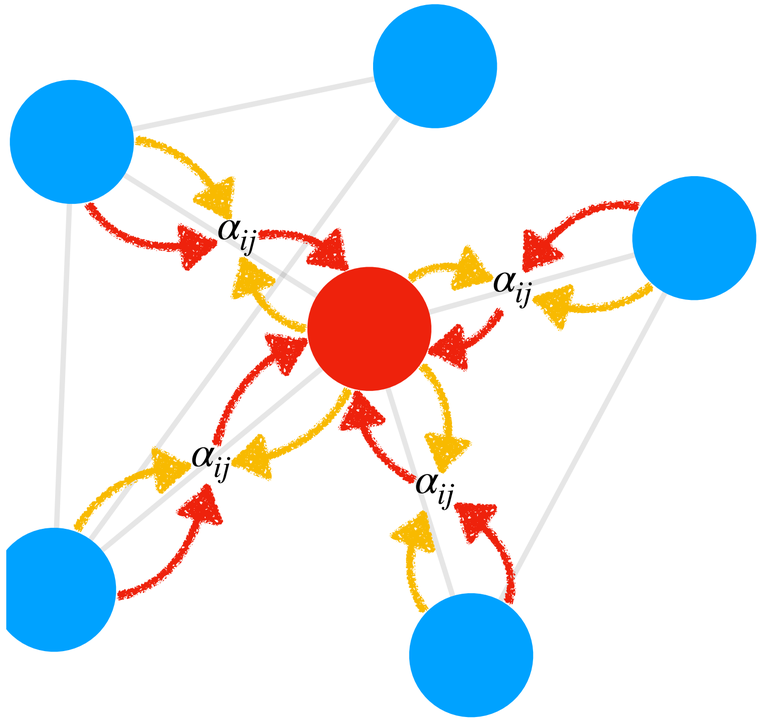
Value: $V = W_V x_j$

Importance weight

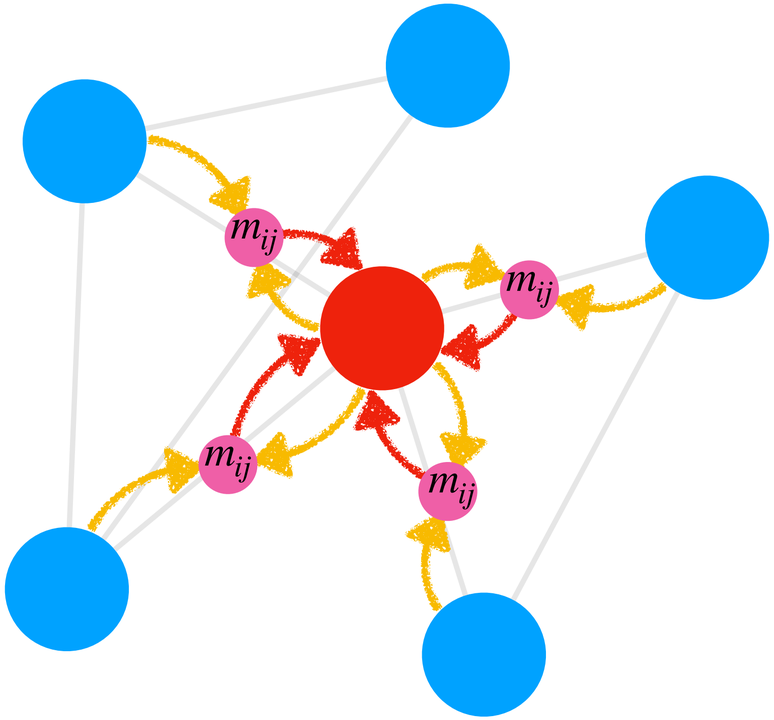
$$\alpha_{ij} = \text{softmax}\left(Q \cdot K / \sqrt{d}\right)$$

Message: $m_{ij} = \alpha_{ij} V$

Attentional



Message passing



Attention

Scaled dot-product attention

Query: $Q = W_Q x_i$

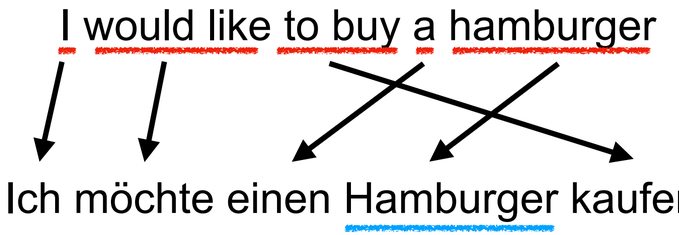
Key: $K = W_K x_j$

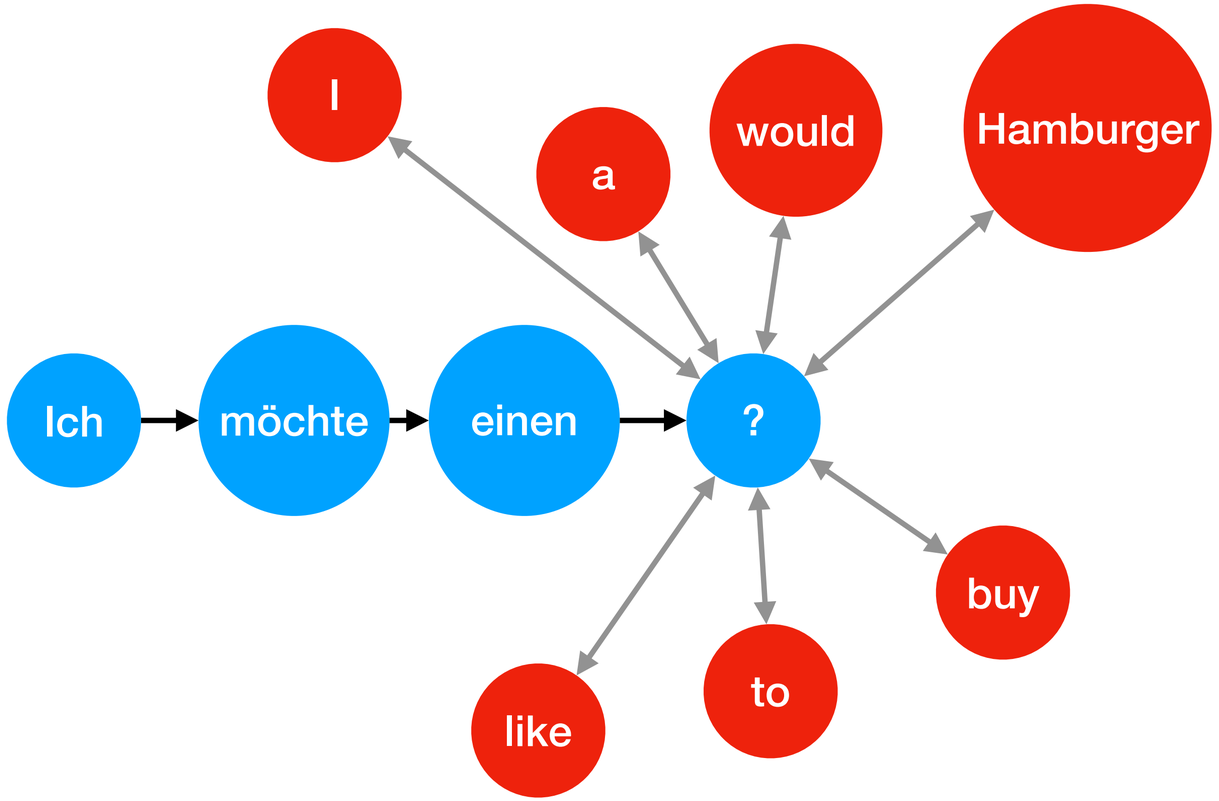
Value: $V = W_V x_j$

Importance weight

$$\alpha_{ij} = \text{softmax}\left(Q \cdot K / \sqrt{d}\right)$$

Message: $m_{ij} = \alpha_{ij} V$







LEARNING TO CONTROL FAST-WEIGHT MEMORIES: AN ALTERNATIVE TO DYNAMIC RECURRENT NETWORKS

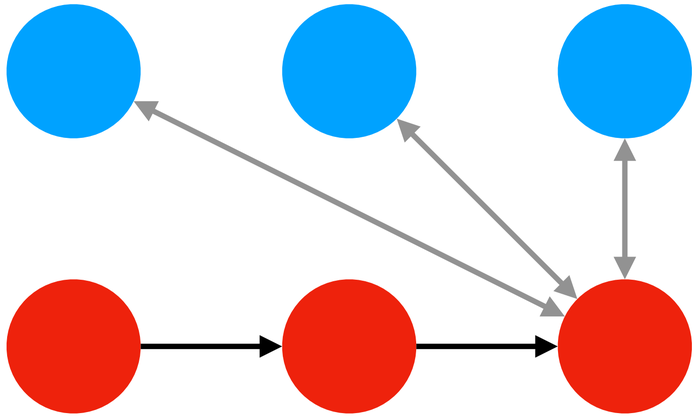
Jürgen Schmidhuber
Institut für Informatik
Technische Universität München
Arcisstr. 21, 8000 München 2, Germany
schmidhu@tumult.informatik.tu-muenchen.de

Technical Report FKI-147-91, March 26, 1991

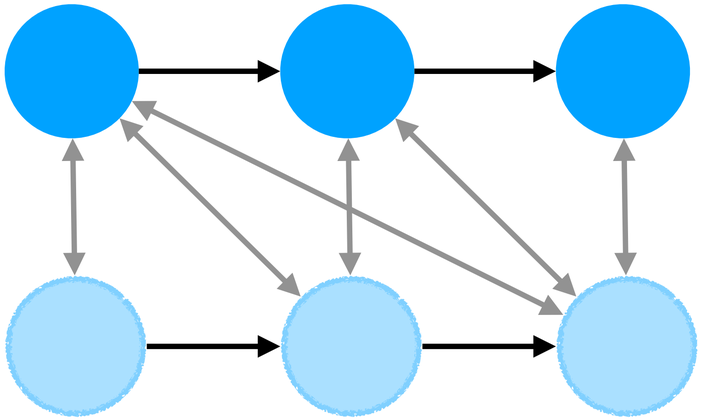
Abstract

Previous algorithms for supervised sequence learning are based on dynamic recurrent networks. This paper describes alternative gradient-based systems consisting of two *feed-forward* nets which learn to deal with temporal sequences by using *fast weights*: The first net *learns* to produce context dependent weight *changes* for the second net whose weights may vary very quickly. The method offers a potential for STM storage efficiency: A simple weight (instead of a full-fledged unit) may be sufficient for storing temporal information. Various learning methods are derived. Two experiments with unknown time delays illustrate the approach. One experiment shows how the system can be used for *adaptive* temporary variable binding.

Cross-attention vs Self-attention



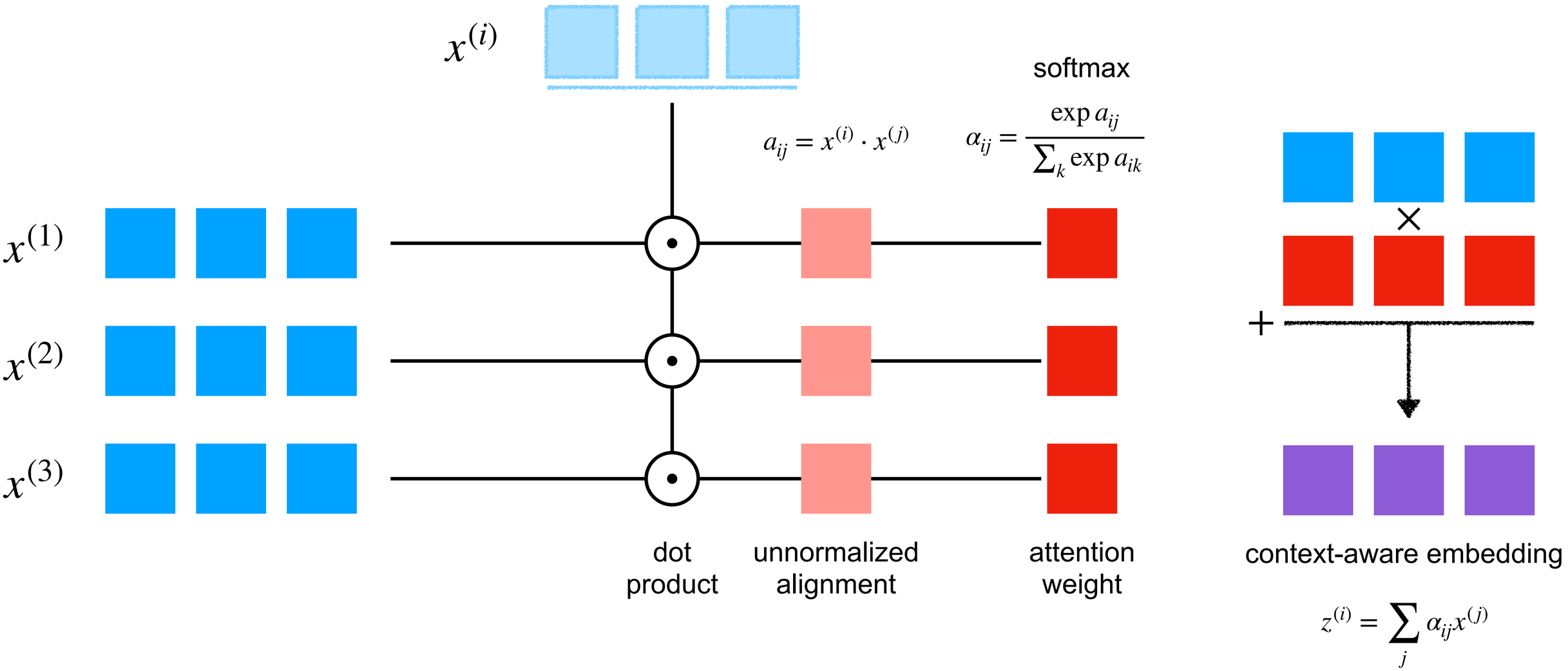
Compute alignment between elements of input and target sequences



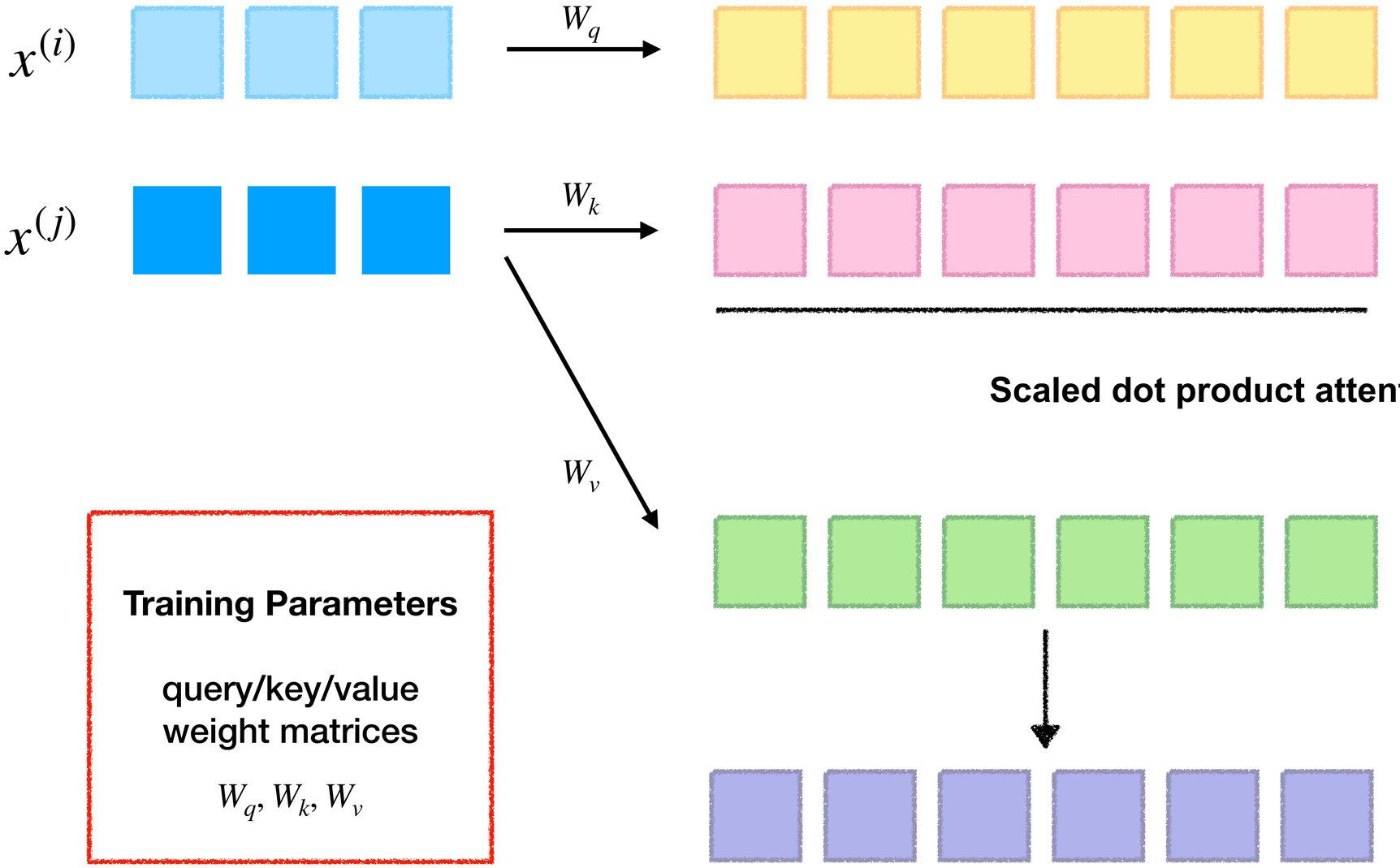
Compute alignment between elements of the same sequence

Context-dependent embedding

Self-attention mechanism



A *trainable* self-attention mechanism



Query $q^{(i)} = W_q\, x^{(i)}$

Key $k^{(j)} = W_k\, x^{(j)}$

Scaled dot product attention $\alpha_{ij} = \text{softmax}\left(\frac{q^{(i)} \cdot k^{(j)}}{\sqrt{d}}\right)$

Value $v^{(j)} = W_v\, x^{(j)}$

Context-aware embedding

$$z^{(i)} = \sum_j \alpha_{ij} v^{(j)}$$

Training Parameters

query/key/value weight matrices

W_q, W_k, W_v

Attention is all you need

ArXiv:1706.03762

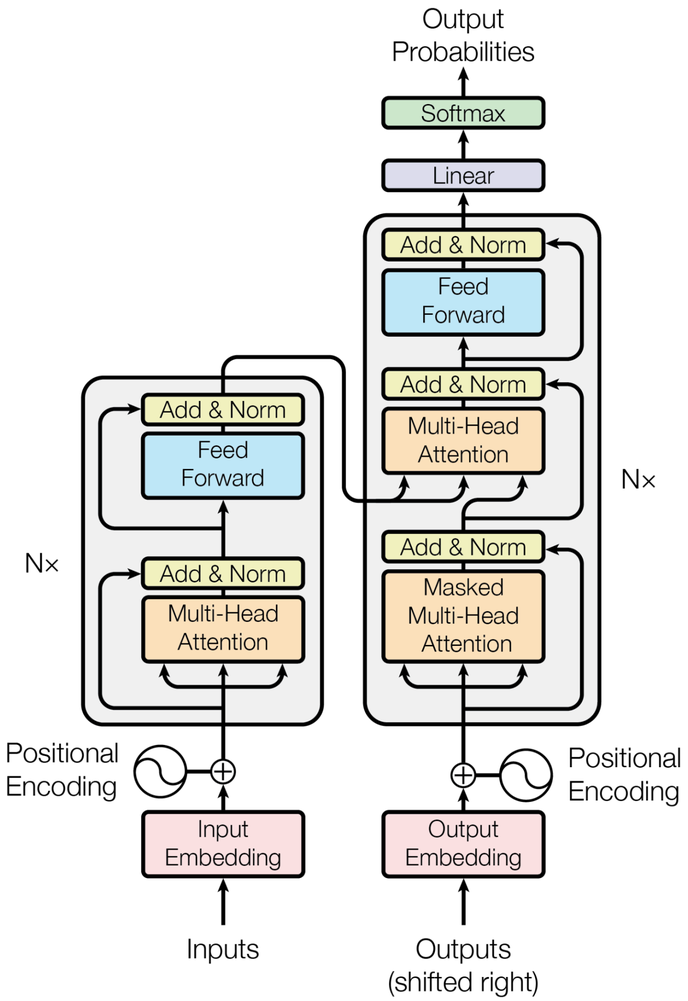


Figure 1: The Transformer - model architecture.

Okay not really

You also need

- Input embedding ✓
- Positional encoding
- Encoder/decoder architecture ✓
- Layer normalization
- Causal masking
- Multi-head attention

Positional encoding

Self-attention processes tokens in parallel, pays no attention to sequence ordering - good or bad?

Positional encoding is added to the embeddings

$$PE_{pos,2i} = \sin \frac{pos}{10{,}000^{2i/d}}$$

$$PE_{pos,2i+1} = \cos \frac{pos}{10{,}000^{2i/d}}$$

pos: position in sequence

i: embedding dimension index

d: embedding dimensionality

Why???

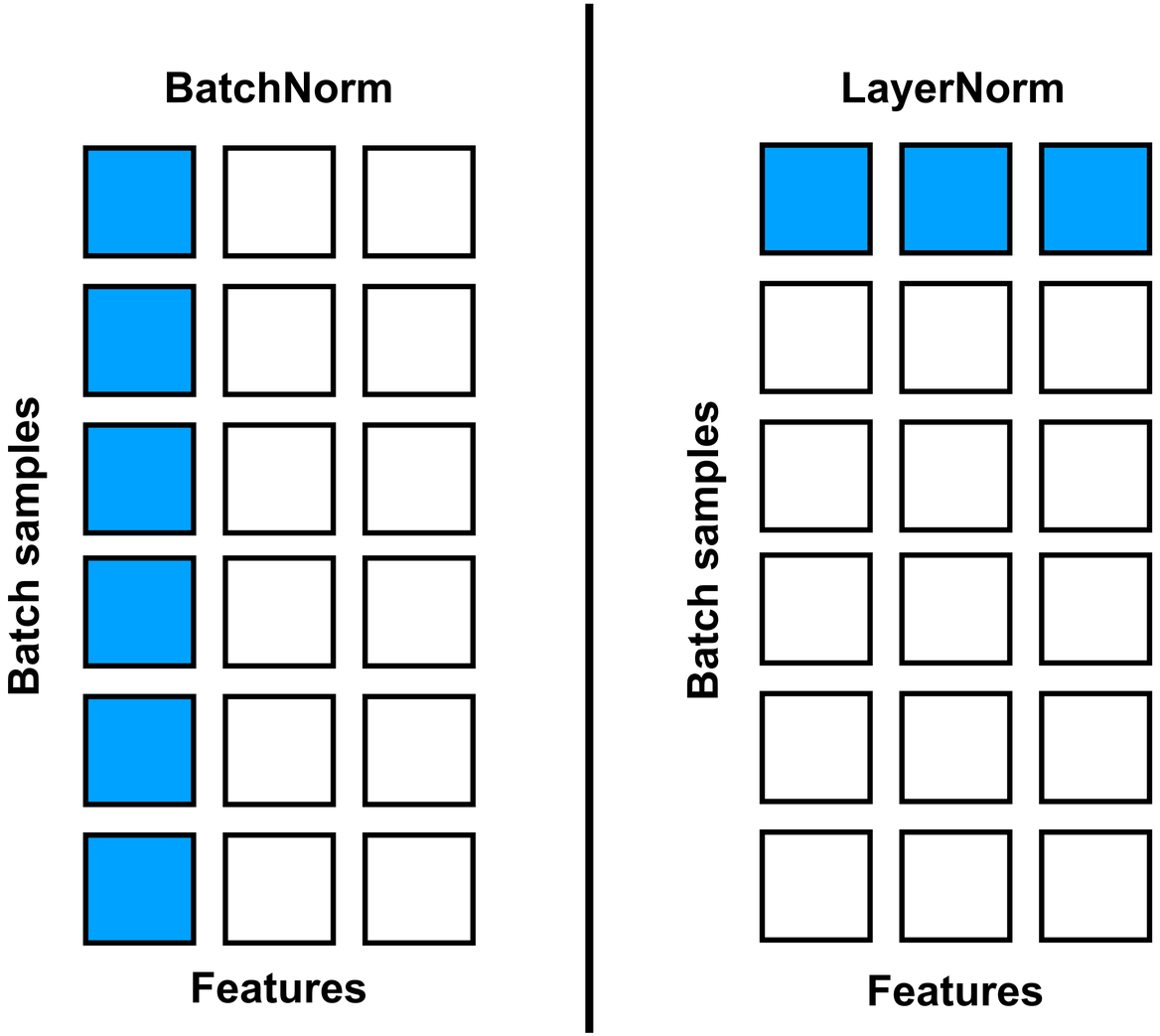
Absolute positional encoding is **unique**

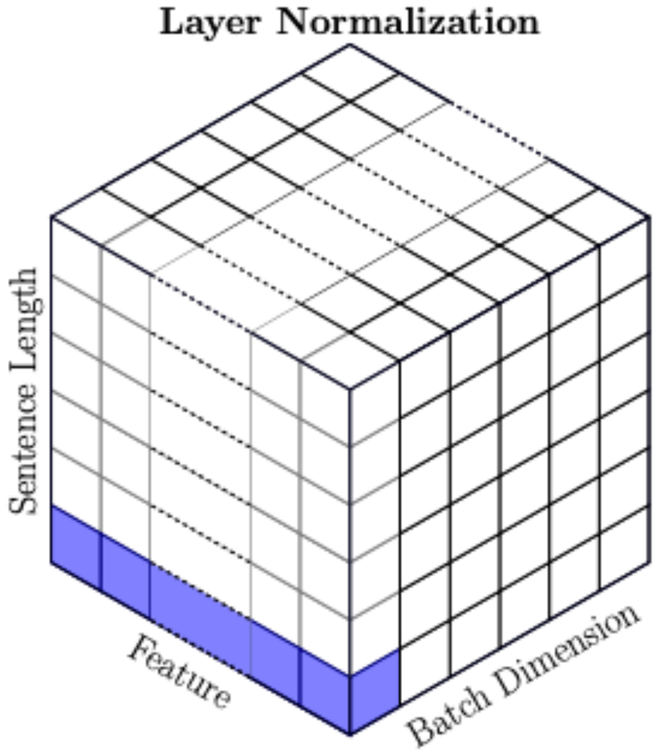
Relative positional encoding is a **linear combination** of inputs

$$\begin{aligned} PE_{pos+k,2} &= \sin \frac{pos+k}{10{,}000^{2/d}} \\ &= \sin \frac{pos}{10{,}000^{2/d}} \cos \frac{k}{10{,}000^{2/d}} + \cos \frac{pos}{10{,}000^{2/d}} \sin \frac{k}{10{,}000^{2/d}} \\ &= c_0^k \, PE_{pos,2} c_0^k + c_1^k \, PE_{pos,3} \end{aligned}$$

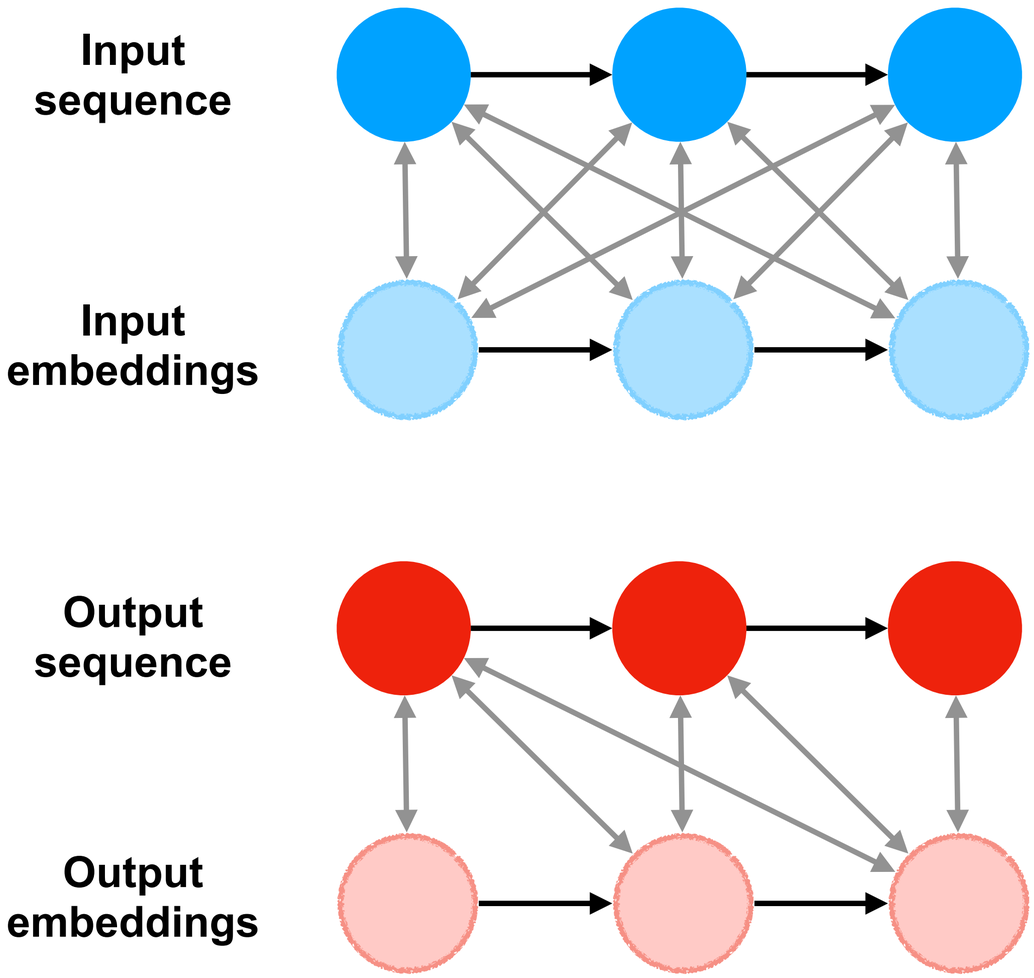
This is expressible via scaled dot product attention

Layer normalization





Causal masking



When encoding, every feature can *self-attend* to features anywhere in the input sequence

When decoding, output features can *cross-attend* to encoded features anywhere in the input sequence

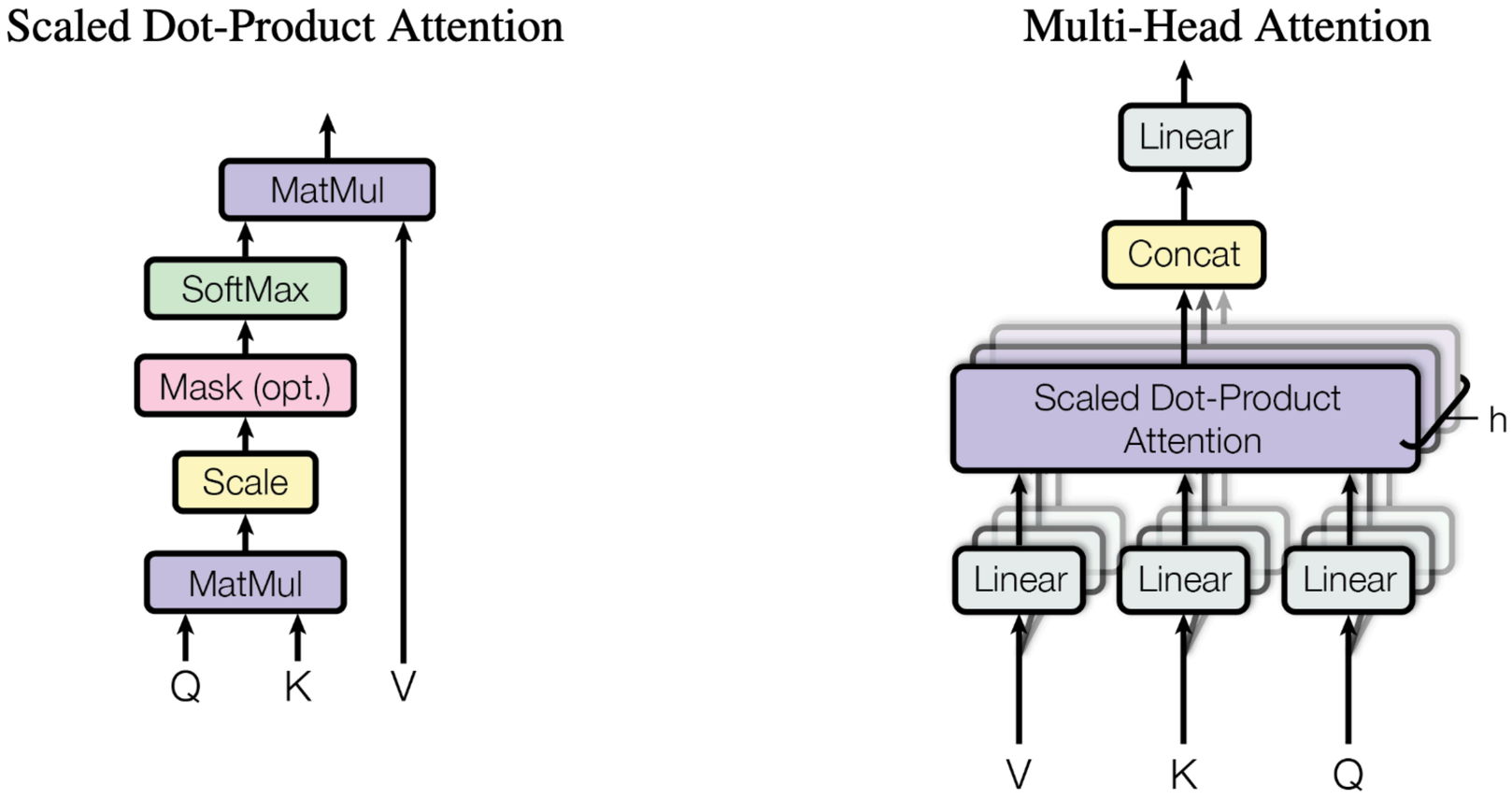
Embeddings leverage *global* context

When decoding, output features *cannot* self-attend to future features in the output sequence

$$\text{MaskedAttention}(q, k, v) = \text{softmax}\left(M + \frac{q \cdot k}{\sqrt{d}}\right) V$$

$$\text{Causal mask: } M = \begin{pmatrix} 0 & -\infty & -\infty & \cdots \\ 0 & 0 & -\infty & \cdots \\ 0 & 0 & 0 & \cdots \end{pmatrix}$$

Multi-head attention



Concurrent attention heads learn distinct embeddings in parallel

Attention is all you need

ArXiv:1706.03762

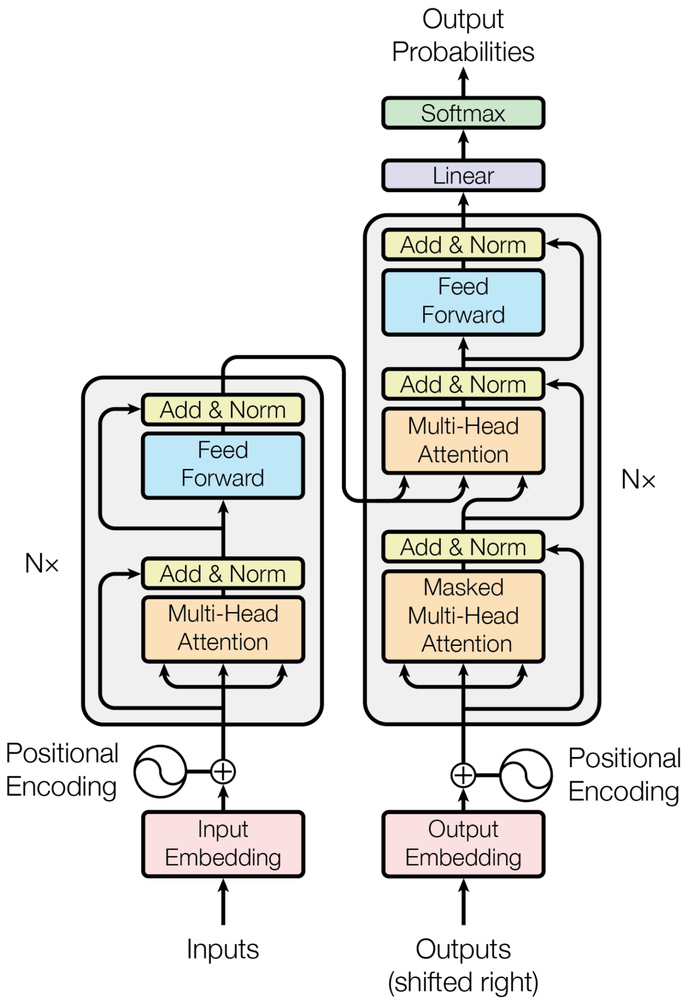


Figure 1: The Transformer - model architecture.

Okay not really

You also need

- Input embedding ✓
- Positional encoding ✓
- Encoder/decoder architecture ✓
- Layer normalization ✓
- Causal masking ✓
- Multi-head attention ✓

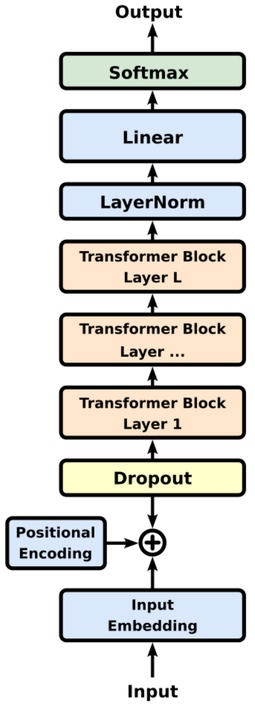
No RNN needed

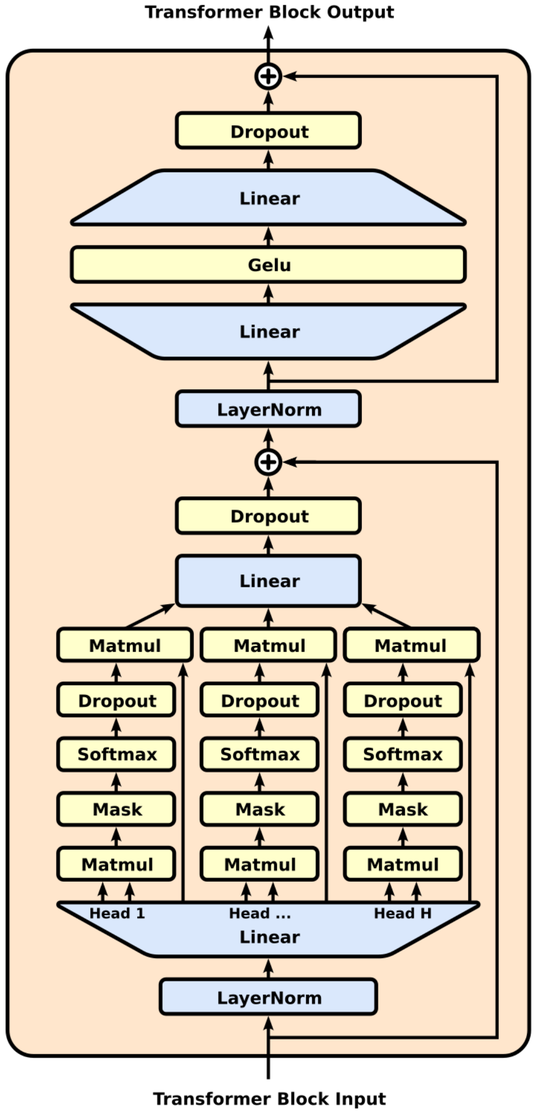
GPT

GPT is a *decoder-only* transformer that learns to predict the next token in a sequence

Causal masking ensures that future information does not bias attention

Generates new sequences autoregressively — consumes the entire input and produced sequence





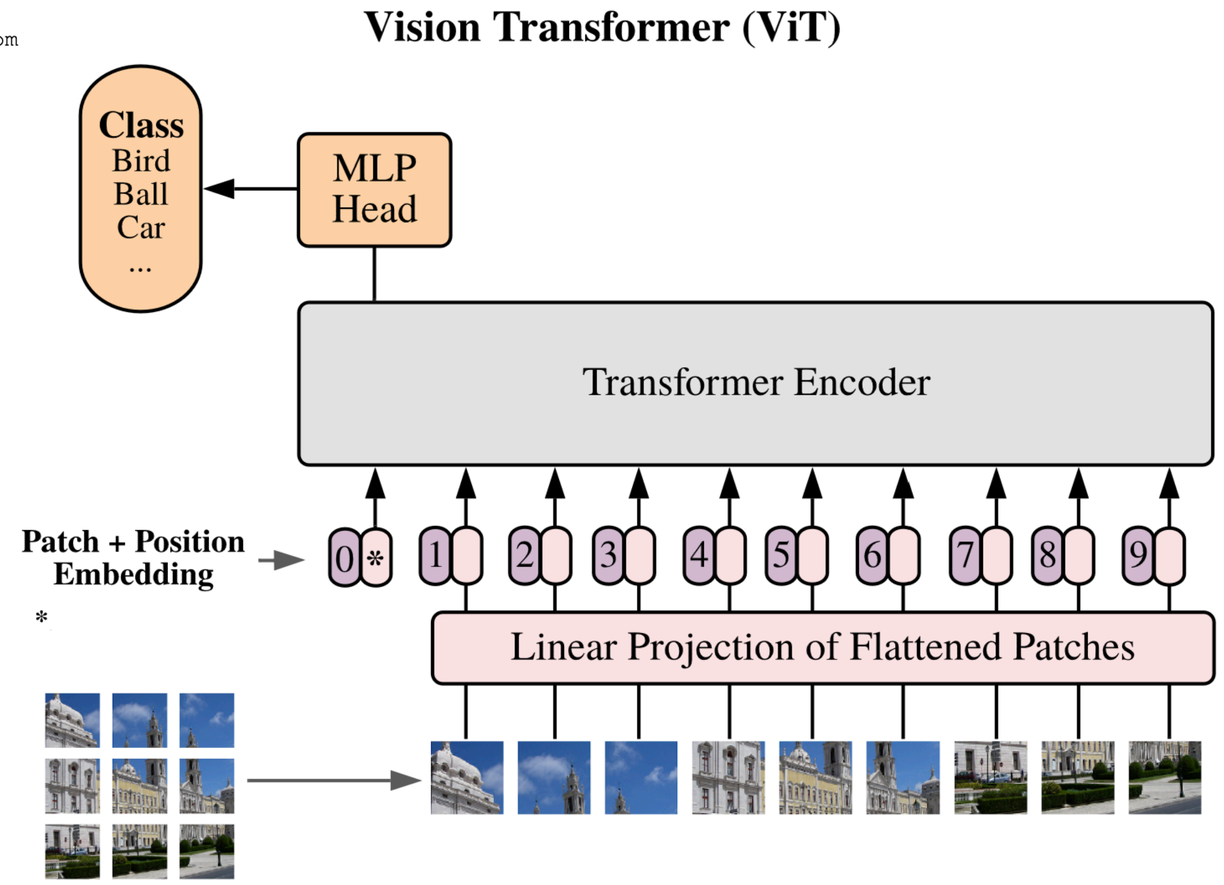
Hands-on with Transformers

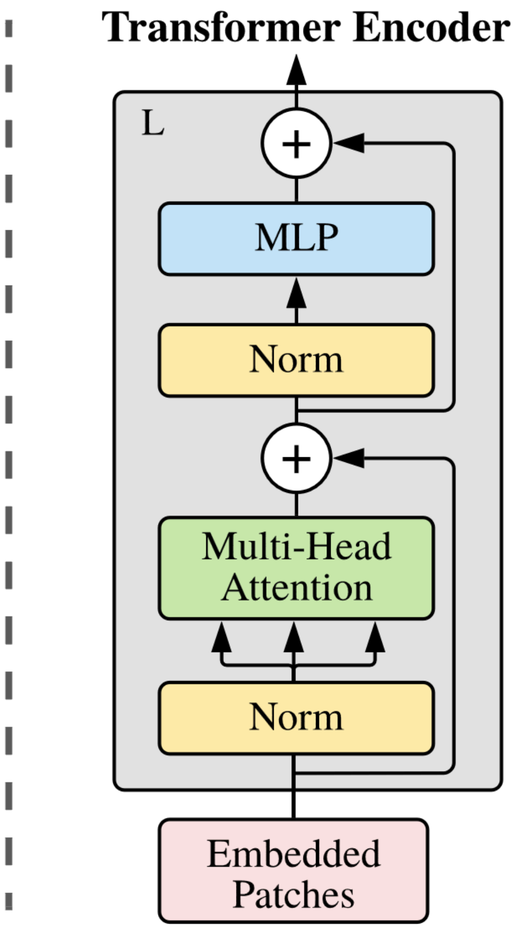


AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

Alexey Dosovitskiy[*,†], **Lucas Beyer**[*], **Alexander Kolesnikov**[*], **Dirk Weissenborn**[*], **Xiaohua Zhai**[*], **Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby**[*,†]
[*]equal technical contribution, [†]equal advising
Google Research, Brain Team
{adosovitskiy, neilhoulsby}@google.com

ArXiv:2010.11929





That’s all!

- Slides will be posted on the course website: https://colen-teaching.github.io/DATA442-Fall-2026
- Lab demos will also be posted
- ***PS3*** due November 8
- **REMEMBER:** TA Office Hours Tuesday 12-1pm, Student Success Center